LAPORAN PRAKTIKUM STRUKTUR DATA

Stack

disusun Oleh:

Tiara Amalia Insani

NIM 2511532019

Dosen Pengampu : DR.Wahyudi, S.T, M.T

Asisten Praktikum : Jovantri Immanuel Gulo

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik dan tepat pada waktunya. Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum pada mata kuliah Struktur Data. Dalam praktikum ini, penulis mempelajari tentang struktur data stack, mulai dari konsep dasar, prinsip kerja *Last In First Out* (LIFO), hingga implementasinya menggunakan bahasa pemrograman Java. Selain itu, praktikum ini juga membahas penerapan stack dalam penyelesaian masalah, seperti evaluasi ekspresi postfix dan pengolahan data. Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penyusunan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Padang, 16 April 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I **1**
PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat 1
BAB II **2**
PEMBAHASAN **2**
2.1 Pengertian Stack 2
2.2 Operasi dalam Stack 2
2.3 Praktikum 2
2.3.1 Alat dan Bahan 2
2.3.2 Program Class stackArray 2
2.3.3 Program Class stackArrayDriver 5
2.3.4 Program Class stackPostfix 7
2.3.5 Program Class NilaiMaksimum 9
2.3.6 Program Class Siswa 11
BAB III **16**
KESIMPULAN DAN SARAN **16**
3.1 Kesimpulan 16
3.2 Saran 16
DAFTAR PUSTAKA **17**
Penjelasan Tugas **18**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, struktur data memiliki peran penting dalam mengatur dan mengelola data agar dapat digunakan secara efisien. Salah satu struktur data dasar yang sering digunakan adalah *stack*. Stack merupakan struktur data yang bekerja dengan prinsip *Last In First Out* (LIFO), yaitu data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Pemahaman tentang stack sangat penting karena konsep ini banyak diterapkan dalam berbagai kasus nyata, seperti pengolahan ekspresi matematika (postfix) dan manajemen memori. mempelajari bagaimana mengimplementasikan stack menggunakan array, serta memanfaatkan struktur data tersebut untuk menyelesaikan permasalahan seperti evaluasi ekspresi postfix dan pencarian nilai maksimum..

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar struktur data stack.
2. Mengetahui prinsip kerja stack dengan metode LIFO (*Last In First Out*).
3. Mengimplementasikan stack menggunakan array dan ArrayList dalam bahasa Java.
4. Menggunakan operasi dasar stack seperti push, pop, dan peek.
5. Menerapkan stack dalam penyelesaian masalah, seperti evaluasi ekspresi postfix.

1.3 Manfaat

Dengan melakukan praktikum, mahasiswa dapat mengetahui bagaimana cara kerja stack dalam menyimpan dan mengambil data, serta memahami penerapannya dalam berbagai permasalahan pemrograman. Selain itu, praktikum ini juga membantu meningkatkan kemampuan logika, ketelitian, dan keterampilan dalam menulis kode program, sehingga mahasiswa lebih siap dalam menghadapi permasalahan yang lebih kompleks di bidang pemrograman dan struktur data.

BAB II

PEMBAHASAN

2.1 Pengertian Stack

Stack merupakan salah satu struktur data yang bersifat linier dan bekerja dengan prinsip *Last In First Out* (LIFO), yaitu elemen yang terakhir dimasukkan ke dalam stack akan menjadi elemen pertama yang dikeluarkan. Stack dapat diibaratkan seperti tumpukan buku, di mana buku yang terakhir diletakkan di atas akan menjadi yang pertama diambil. Dalam implementasinya, stack memiliki satu akses utama yaitu pada bagian atas (*top*), sehingga proses penambahan maupun penghapusan data hanya dapat dilakukan pada bagian tersebut. Stack banyak digunakan dalam berbagai aplikasi, seperti pengolahan ekspresi matematika, manajemen memori, serta fitur undo dan redo pada perangkat lunak.

2.2 Operasi dalam Stack

a) Push → untuk menambahkan elemen ke dalam stack. Elemen yang ditambahkan akan ditempatkan pada posisi paling atas (*top*) dari stack.
b) Pop → untuk menghapus elemen dari stack. Elemen yang dihapus adalah elemen yang berada di posisi paling atas sesuai dengan prinsip LIFO.
c) Peek → untuk melihat elemen yang berada di posisi paling atas tanpa menghapusnya dari stack.

2.3 Praktikum

2.3.1 Alat dan Bahan

1. Laptop/computer dengan eclipse terinstal.
2. Materi praktikum mengenai Array List

2.3.2 Program Class stackArray

```
package Pekan3_2511532019;

public class stackArray_2511532019 {
    static final int MAX_2019=1000;
    int top;
    int a_2019[] = new int [MAX_2019];
    boolean isEmpty_2019() {
          return (top<0);
    }
```

```
    stackArray_2511532019 (){
        top=-1;
    }
    boolean push_2019 (int x)
    {
        if (top>= (MAX_2019-1)) {
            System.out.println("Stack Overlow");
            return false;
        }
        else {
            a_2019[++top]=x;
            System.out.println(x+" dimasukkan dalam stack");
            return true;
        }
    }
    int pop_2019() {
        if (top<0) {
            System.out.println("Stack Underflow");
            return 0;
        }else {
            int x=a_2019[top--];
            return x;
        }
    }
    int peek_2019 ()
    {
        if (top <0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a_2019 [top];
            return x;
        }
    }
    void print_2019() {
        for (int i = top; i>-1; i--) {
            System.out.print(" "+a_2019[i]);
        }
    }
}
```

1. `package Pekan3_2511532019;`

   Menentukan package tempat class ini disimpan

2. `public class stackArray_2511532019 {`

   Membuat class Bernama stackArray

3. `static final int MAX_2019=1000;`

   Membuat konstatnta max_2019=ukuran maksimal stack (1000 data)

   Static final = nilainya tetap tidak bisa diubah

4. `int top;`

   Variabel top untuk menyimpan posisi data paling atas di satck

5. `int a_2019[] = new int [MAX_2019];`

   membuat array Bernama a_2019 dengan ukuran 1000 untuk menyimpan data stack

6. `boolean isEmpty_2019() {`

   Method untuk mengecek apakah stack kosong

7. `return (top<0);`

   jika top<0, berarti stack kosong, return true

8. `stackArray_2511532019 (){`

   konstruktor

9. `top=-1;`

   Mengatur stack awalnya kosong (top= -1)

10. `boolean push_2019 (int x)`

    Method untuk menambahkan data (push) ke stack

11. `if (top>= (MAX_2019-1)) {`

    cek apakah stack sudah penuh

12. `System.out.println("Stack Overlow");`

    Jika penuh, tampilkan pesan “stack overflow” (kelebihan dara)

13. `return false;`

    gagal menambahkan data

14. `else {`

    jika stack belum penuh

15. `a_2019[++top]=x;`

    tambahkan data ke stack, top dinaikkan dulu, lalu isi array dengan nilai x

16. `System.out.println(x+" dimasukkan dalam stack");`

    Menampilkan pesan bahwa data berhasil dimasukkan

17. `return true;`

    berhasil push

18. `int pop_2019() {`

    method untuk menghapus data paling atas (pop)

19. `if (top<0) {`

    cek apakah stack kosong

20. `System.out.println("Stack Underflow");`

    Jika kosong tampilkan pesan error

21. `return 0;`

    mengembalikan nilai nol (karena tidak ada data)

22. `}else {`

    Jika tidak kosong

23. `int x=a_2019[top--];`

    ambil data paling atas, lalu top dikurangi

24. `return x;`

    mengembalikan data yang dihapus

25. `int peek_2019 ()`

    method untuk melihat data paling atas tanpa menghapusya

26. `if (top <0) {`

    cek apakah kosong

27. `System.out.println("Stack Underflow");`

    Jika kosong tampilkan error

28. `return 0;`

    menegmbalikan nilai nol (tidak ada data)

29. `else {`

    jika ada data

30. `int x = a_2019 [top];`

    ambil data paling atas tampa menghapus

31. `return x;`

    kembalikan nilainya

32. `void print_2019() {`

    method untuk menampilkan nilai stack

33. `for (int i = top; i>-1; i--) {`

    loop dari atas ke bawah stack

34. `System.out.print(" "+a_2019[i]);`

    Cetak setiap isi stack

2.3.3 Program Class stackArrayDriver

```
package Pekan3_2511532019;
```

```
public class stackArrayDriver_2511532019 {
    public static void main (String[] args) {
            stackArray_2511532019 s = new stackArray_2511532019();
            s.push_2019(10);
            s.push_2019(20);
            s.push_2019(30);
            System.out.println(s.pop_2019()+" dikeluarkan dari stack");
            System.out.println("Elemen teratas adalah : " +
s.peek_2019());
            System.out.print("Elemen pada stack     :");
            s.print_2019();
    }
}
```

1. `package Pekan3_2511532019;`

   Menentukan package (folder) tempat class ini disimpan

2. `public class stackArrayDriver_2511532019 {`

   Mendeklarasikan class utama untuk menjalankan program

3. `public static void main (String[] args) {`

   method utama main, program mulai dari sini

4. `stackArray_2511532019 s = new stackArray_2511532019();`

   membuat objek s dari class stackArray

5. `s.push_2019(10);`

   menambahkan angka 10 ke stack

6. `s.push_2019(20);`

   menambahkan angka 20 ke stack

7. `s.push_2019(30);`

   menambahkan angka 30 ke stack

8. `System.out.println(s.pop_2019()+" dikeluarkan dari stack");`

   Menghapus (pop) data paling atas, yang keluar=30

9. `System.out.println("Elemen teratas adalah : " + s.peek_2019());`

   Melihat data paling atas tanpa menghapus, yaitu 20

10. `System.out.print("Elemen pada stack     :");`

    Menampilkan teks sebelum isi stack

11. `s.print_2019();`

    menampilkan semua isi stack dari atas ke bawah

Output

```
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah : 20
Elemen pada stack     : 20 10
```

2.3.4 Program Class stackPostfix

```
package Pekan3_2511532019;

import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511532019 {
    public static int postfixEvaluate_2019 (String expression) {
            Stack <Integer> s_2019 = new Stack <Integer>();
            Scanner input_2019 = new Scanner (expression);
            while (input_2019.hasNext()) {
                    if (input_2019.hasNextInt()) {
                            s_2019.push(input_2019.nextInt());
                    }else {
                            String operator_2019 = input_2019.next();
                            int operand2_2019=s_2019.pop();
                            int operand1_2019=s_2019.pop();
                            if (operator_2019.equals("+")) {
                                    s_2019.push
(operand1_2019+operand2_2019);
                            }else if (operator_2019.equals("-")) {
                                    s_2019.push(operand1_2019-
operand2_2019);
                            }else if (operator_2019.equals("*")) {

    s_2019.push(operand1_2019*operand2_2019);
                            }else {

    s_2019.push(operand1_2019/operand2_2019);
                            }
                    }
            }
            input_2019.close();
            return s_2019.pop();
    }
    public static void main (String [] args) {
            System.out.println("hasil postfix = "+ postfixEvaluate_2019
("5 2 4 * + 7 -"));
    }
}
```

1. `package Pekan3_2511532019;`

   menentukan peckage tempat class ini disimpan

2. `import java.util.Scanner;`

mengimpor scanner untuk membaca input

3. ```
   import java.util.Stack;
   ```
   mengimpor class stack

4. ```
   public class StackPostfix_2511532019 {
   ```
   membuat class untuk menghitung ekspresi postfix

5. ```
   public static int postfixEvaluate_2019 (String expression) {
   ```
   method untuk menghitung nilai dari postfix expression

6. ```
   Stack <Integer> s_2019 = new Stack <Integer>();
   ```
   Membuat stack kosong utuk menyimpan angka

7. ```
   Scanner input_2019 = new Scanner (expression);
   ```
   Membaca string expression satu per satu

8. ```
   while (input_2019.hasNext()) {
   ```
   selama masih ada data yang bisa dibaca

9. ```
   if (input_2019.hasNextInt()) {
   ```
   jika yang dibaca adalah angka

10. ```
    s_2019.push(input_2019.nextInt());
    ```
    masukkan angka ke dalam stack

11. ```
    }else {
    ```
    Jika bukan angka

12. ```
    String operator_2019 = input_2019.next();
    ```
    Ampil operatornya

13. ```
    int operand2_2019=s_2019.pop();
    ```
    ambil angka kedua dari stack (yang paling atas)

14. ```
    int operand1_2019=s_2019.pop();
    ```
    ambil angka pertama

15. ```
    if (operator_2019.equals("+")) {
    ```
    jika operatornya +

16. ```
    s_2019.push (operand1_2019+operand2_2019);
    ```
    hitung penjumlahan lalu simpan hasilnya ke stack

17. ```
    }else if (operator_2019.equals("-")) {
    ```
    Jika operatornya -

18. ```
    s_2019.push(operand1_2019-operand2_2019);
    ```
    kurangkan, lalu simpan hasilnya ke stack

19. `}else if (operator_2019.equals("*")) {`

    Jika operatornya *

20. `s_2019.push(operand1_2019*operand2_2019);`

    kalikan, lalu simpan hasilnya ke stack

21. `}else {`

    Jika buka (+,-,*) berarti /

22. `s_2019.push(operand1_2019/operand2_2019);`

    bagi, lalu simpan hasilnya ke stack

23. `input_2019.close();`

    menutup scanner

24. `return s_2019.pop();`

    menagmbil hasil akhir dari stack

25. `public static void main (String [] args) {`

    method utama program

26. `System.out.println("hasil postfix = "+ postfixEvaluate_2019 ("5 2 4 * + 7 -"));`

    Menghituung ekspresi postfix

    Output

```
hasil postfix = 6
```

2.3.5 Program Class NilaiMaksimum

```
package Pekan3_2511532019;
import java.util.Stack;
public class NilaiMaksimum_2511532019 {
    public static int max_2019(Stack<Integer>s_2019) {
            Stack<Integer> backup_2019 = new Stack <Integer>();
            int maxValue_2019= s_2019.pop();
            backup_2019.push(maxValue_2019);
            while(!s_2019.isEmpty()) {
                    int next_2019 = s_2019.pop();
                    backup_2019.push(next_2019);
                    maxValue_2019=Math.max(maxValue_2019, next_2019);
            }
            while (!backup_2019.isEmpty()) {
                    s_2019.push(backup_2019.pop());
                    }
            return maxValue_2019;
            }
    public static void main (String[] args) {
            Stack <Integer> s_2019= new Stack <Integer>();
            s_2019.push(70);
            s_2019.push(12);
```

```
        s_2019.push(20);
        System.out.println("isi stack " +s_2019);
        System.out.println("Stack Teratas "+s_2019.peek());
        System.out.println("Nilai maksimum "+ max_2019 (s_2019));
    }
}
```

1. `package Pekan3_2511532019;`
2. menentukan peckage tempat class ini disimpan
3. `import java.util.Stack;`

   mengimpor class stack
4. `public class NilaiMaksimum_2511532019 {`

   membuat class untuk mencari nilai maksimum
5. `public static int max_2019(Stack<Integer>s_2019) {`

   method untuk mencari nilai terbesar dari stack
6. `Stack<Integer> backup_2019 = new Stack <Integer>();`

   Membuat stack Cadangan supaya data asli tidak hilang
7. `int maxValue_2019= s_2019.pop();`

   ambil data paling atas sebagai nilai awal maksimum
8. `backup_2019.push(maxValue_2019);`

   simpan data tadi ke stack backup
9. `while(!s_2019.isEmpty()) {`

   selama stack utama belum kosong
10. `int next_2019 = s_2019.pop();`

    ambil satu persatu data dari stack
11. `backup_2019.push(next_2019);`

    simpan ke backup supaya nanti bisa dikembalikan
12. `maxValue_2019=Math.max(maxValue_2019, next_2019);`

    bandingkan nilai sekarang denagn nilai sebelumnya, ambil yang paling besar
13. `while (!backup_2019.isEmpty()) {`

    selama backup masih ada isi
14. `s_2019.push(backup_2019.pop());`

    kembalikan semua data ke stack asli
15. `return maxValue_2019;`

    mengambalikan nilai maksimum

16. `public static void main (String[] args) {`

    method utama program

17. `Stack <Integer> s_2019= new Stack <Integer>();`

    Membuat stack kosong

18. `s_2019.push(70);`

    `s_2019.push(12);`

    `s_2019.push(20);`

    menambahkan data ke stack

19. `System.out.println("isi stack " +s_2019);`

    Menampilkan isi stack

20. `System.out.println("Stack Teratas "+s_2019.peek());`

    Menampilkan elemen paling atas

21. `System.out.println("Nilai maksimum "+ max_2019 (s_2019));`

    Memanggil method untuk mencari nilai terbesar

    Output

```
isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

2.3.6 Program Class Siswa

```
package Pekan3_2511532019;

import java.util.ArrayList;
class Siswa_2019{
    String nama_2019;
    int nim_2019;

    public Siswa_2019 (String nama_2019, int nim_2019) {
            this.nama_2019=nama_2019;
            this.nim_2019=nim_2019;
    }
    @Override
    public String toString() {
            return "Nim : "+nim_2019+", nama: " +nama_2019;
    }
}

public class SiswaStack_2511532019 {
    private ArrayList<Siswa_2019>stack_2019;
    public SiswaStack_2511532019() {
            stack_2019 = new ArrayList<>();
    }
    public void push_2019 (Siswa_2019 mhs) {
```

```
            stack_2019.add(mhs);
    }

    public Siswa_2019 pop_2019() {
            if (!isEmpty()) {
                    return stack_2019.remove(stack_2019.size()-1);
            }
            return null;
    }

    public Siswa_2019 peek_2019() {
            if(!isEmpty()) {
                    return stack_2019.get(stack_2019.size()-1);
            }
            return null;
    }
    public boolean isEmpty() {
            return stack_2019.isEmpty();
    }
    public void tampilkanSiswa() {
            for (int i=stack_2019.size()-1;i>=0;i--) {
                    System.out.println(stack_2019.get (i));
            }
    }
    public static void main (String[] args) {
            SiswaStack_2511532019 studentStack_2019=new
SiswaStack_2511532019();
            Siswa_2019 mhs1=new Siswa_2019 ("Ali", 1);
            Siswa_2019 mhs2=new Siswa_2019 ("Boby",2);
            Siswa_2019 mhs3=new Siswa_2019 ("Charles",3);
            studentStack_2019.push_2019 (mhs1);
            studentStack_2019.push_2019(mhs2);
            studentStack_2019.push_2019(mhs3);
            System.out.println("Siswa di dalam stack:");
            studentStack_2019.tampilkanSiswa();
            System.out.println("siswa teratas
"+studentStack_2019.peek_2019());
            System.out.println("mengeluarkan siswa teratas dari stack:
"+studentStack_2019.pop_2019());
            System.out.println("daftar siswa setelah di pop :");
            studentStack_2019.tampilkanSiswa();
    }

}
```

1. `package Pekan3_2511532019;`
2. menentukan peckage tempat class ini disimpan
3. `import java.util.ArrayList;`

   mengimpor arrayList agar bisa digunakan
4. `class Siswa_2019{`

   membuat class untuk menyimpan data siswa

5. `String nama_2019;`
   `int nim_2019;`
   atribut
6. `public Siswa_2019 (String nama_2019, int nim_2019) {`
   konstruktor untuk mengisi saat objek dibuat
7. `this.nama_2019=nama_2019;`
   `this.nim_2019=nim_2019;`
   mengisi nilai ke dalam atribut
8. `public String toString() {`
   method untuk megatur tampilan objek saat dicetak
9. `return "Nim : "+nim_2019+", nama: " +nama_2019;`
   format output siswa
10. `public class SiswaStack_2511532019 {`
    class utama untuk membuat stack berisi objek siswa
11. `private ArrayList<Siswa_2019>stack_2019;`
    menggunakan ArrayList sebagai tempat penyimpanan data stack
12. `public SiswaStack_2511532019() {`
    konstruktor
13. `stack_2019 = new ArrayList<>();`
    membuat ArrayList kosong
14. `public void push_2019 (Siswa_2019 mhs) {`
    method untuk menambah siswa ke stack
15. `stack_2019.add(mhs);`
    tambahkan ke belakang
16. `public Siswa_2019 pop_2019() {`
    method untuk mengambil dan menghapus data teratas
17. `if (!isEmpty()) {`
    cek kalau stack tidak kosong
18. `return stack_2019.remove(stack_2019.size()-1);`
    ambil dan hapus elemen terakhir
19. `return null;`
    jika kosong, kembalikan null
20. `public Siswa_2019 peek_2019() {`
    method untuk melihat data teratas tanpa menghapus

21. `if(!isEmpty()) {`

    cek tidak kosong

22. `return stack_2019.get(stack_2019.size()-1);`

    ambil elemen terakhir

23. `return null;`

    jika kosong, kembalikan null

24. `public boolean isEmpty() {`

    mengecek apakah stack kosong

25. `return stack_2019.isEmpty();`

    return true jika kosong

26. `public void tampilkanSiswa() {`

    method untuk menampilkan semua siswa dalam stack

27. `for (int i=stack_2019.size()-1;i>=0;i--) {`

    loop dari atas ke bawah

28. `System.out.println(stack_2019.get (i));`

    Cetak data siswa

29. `public static void main (String[] args) {`

    method utama program

30. `SiswaStack_2511532019 studentStack_2019=new SiswaStack_2511532019();`

    Membuat stack siswa

31. `Siswa_2019 mhs1=new Siswa_2019 ("Ali", 1);`
    `Siswa_2019 mhs2=new Siswa_2019 ("Boby",2);`
    `Siswa_2019 mhs3=new Siswa_2019 ("Charles",3);`

    Membuat 3 objek siswa

32. `studentStack_2019.push_2019 (mhs1);`
    `studentStack_2019.push_2019(mhs2);`
    `studentStack_2019.push_2019(mhs3);`

    masukkan ke stack

33. `System.out.println("Siswa di dalam stack:");`
    `studentStack_2019.tampilkanSiswa();`

    menampilkan semua siswa

34. `System.out.println("siswa teratas "+studentStack_2019.peek_2019());`

    Menampilkan siswa paling atas

35. ```
System.out.println("mengeluarkan siswa teratas dari stack: "+studentStack_2019.pop_2019());
```

Menghapus siswa teratas

36. ```
System.out.println("daftar siswa setelah di pop :");
studentStack_2019.tampilkanSiswa();
```

Menampilkan isi stack setelah pop

Output

```
Siswa di dalam stack:
Nim : 3, nama: Charles
Nim : 2, nama: Boby
Nim : 1, nama: Ali
siswa teratas Nim : 3, nama: Charles
mengeluarkan siswa teratas dari stack: Nim : 3, nama: Charles
daftar siswa setelah di pop :
Nim : 2, nama: Boby
Nim : 1, nama: Ali
```

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data stack merupakan struktur data linier yang bekerja dengan prinsip *Last In First Out* (LIFO), di mana elemen terakhir yang dimasukkan akan menjadi elemen pertama yang dikeluarkan. Stack memiliki beberapa operasi dasar, yaitu *push*, *pop*, *peek*, serta pengecekan kondisi seperti *isEmpty* dan *isFull*.

Dalam praktikum ini, implementasi stack dilakukan menggunakan array dan ArrayList dalam bahasa pemrograman Java. Selain itu, stack juga dapat digunakan untuk menyelesaikan berbagai permasalahan, seperti evaluasi ekspresi postfix dan pencarian nilai maksimum. Dengan memahami konsep dan implementasinya, mahasiswa dapat mengetahui bagaimana stack bekerja serta penerapannya dalam pemrograman.

3.2 Saran

1. lebih banyak berlatih dalam mengimplementasikan stack agar semakin memahami konsepnya
2. Perlu meningkatkan ketelitian dalam penulisan kode untuk menghindari kesalahan (*error*) saat program dijalankan.

DAFTAR PUSTAKA

[1] M. T. Goodrich, R. Tamassia, dan M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ: Wiley, 2014.

Penjelasan Tugas

Program Class Website

```java
package Pekan3_2511532019;

public class Website_2511532019 {
       private String judul_2019;
    private String url_2019;

    // Constructor
    public Website_2511532019(String judul_2019, String url_2019) {
        this.judul_2019 = judul_2019;
        this.url_2019 = url_2019;
    }

    // Getter
    public String getJudul_2019() {
        return judul_2019;
    }

    public String getUrl_2019() {
        return url_2019;
    }

    // Setter
    public void setJudul_2019(String judul_2019) {
        this.judul_2019 = judul_2019;
    }

    public void setUrl_2019(String url_2019) {
        this.url_2019 = url_2019;
    }

    @Override
    public String toString() {
        return "Judul: " + judul_2019 + " | URL: " + url_2019;
    }
}
```

1. `package Pekan3_2511532019;`

   menentukan peckage tempat class disimpan

2. `public class Website_2511532019 {`

   membuat class Bernama website

3. `private String judul_2019;`

   variavel untuk menyimpan judul website

4. `private String url_2019;`

   variabel untuk menyimpan Alamat URL website

5. `public Website_2511532019(String judul_2019, String url_2019) {`

konstruktor, method yang otomatis dipanggil saat objek dibuat, digunakan untuk mengisi nilai awal.

6. `this.judul_2019 = judul_2019;`
   mengisi variabel judul di dalam class dengan nilai dari parameter
7. `this.url_2019 = url_2019;`
   mengisi variavel URL
8. `public String getJudul_2019() {`
   method untuk mengambil nilai judul
9. `return judul_2019;`
   mengembalikan isi variabel judul
10. `public String getUrl_2019() {`
    method untuk mengambil URL
11. `return url_2019;`
    mengembalikan nilai URL
12. `public void setJudul_2019(String judul_2019) {`
    method untuk mengubah nilai judul
13. `this.judul_2019 = judul_2019;`
    mengganti isi variabel judul dengan nilai baru
14. `public void setUrl_2019(String url_2019) {`
    method untuk mengubah URL
15. `this.url_2019 = url_2019;`
    mengganti isi URL
16. `public String toString() {`
    method untuk menentuka tampilan objek saat dicetak
17. `return "Judul: " + judul_2019 + " | URL: " + url_2019;`
    mengatur format output

Program Class Browser

```
package Pekan3_2511532019;

import java.util.Scanner;
import java.util.Stack;

public class Browser_2511532019 {
      public static void main(String[] args) {
        Stack<Website_2511532019> history_2019 = new Stack<>();
        Scanner input_2019 = new Scanner(System.in);
```

```java
        int pilihan_2019;

        do {
            System.out.println("\n=== Browser History NIM: 2511532019
===");
            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");
            System.out.println("3. Lihat Halaman Aktif (Peek)");
            System.out.println("4. Cek Status History");
            System.out.println("5. Keluar");
            System.out.print("Pilihan: ");
            pilihan_2019 = input_2019.nextInt();
            input_2019.nextLine();

            switch (pilihan_2019) {
            case 1:
                System.out.print("Masukkan Judul: ");
                String judul_2019 = input_2019.nextLine();

                System.out.print("Masukkan URL: ");
                String url_2019 = input_2019.nextLine();

                Website_2511532019 web = new
Website_2511532019(judul_2019, url_2019);
                history_2019.push(web);

                System.out.println("Berhasil mengunjungi halaman!");
                break;

            case 2:
                if (!history_2019.isEmpty()) {
                    Website_2511532019 removed = history_2019.pop();
                    System.out.println("Halaman dihapus: " +
removed.getJudul_2019());
                } else {
                    System.out.println("History kosong!");
                }
                break;

            case 3:
                if (!history_2019.isEmpty()) {
                    System.out.println("Halaman aktif: " +
history_2019.peek());
                } else {
                    System.out.println("Tidak ada halaman aktif!");
                }
                break;

            case 4:
                if (history_2019.isEmpty()) {
                    System.out.println("History kosong!");
                } else {
                    System.out.println("Jumlah history: " +
history_2019.size());
                }
                break;
```

```
            case 5:
                System.out.println("Keluar dari program...");
                break;

            default:
                System.out.println("Pilihan tidak valid!");
        }

    } while (pilihan_2019 != 5);

    input_2019.close();
}
}
```

1. `package Pekan3_2511532019;`

   menentukan package tempat class disimpan

2. `import java.util.Scanner;`

   digunakan untuk input dari keyboard

3. `import java.util.Stack;`

   digunakan untuk memakai struktur data stack

4. `public class Browser_2511532019 {`

   membuat class Bernama Browser

5. `public static void main(String[] args) {`

   method utama

6. `Stack<Website_2511532019> history_2019 = new Stack<>();`

   Membuat stack Bernama history_2019, isi stack adalah website_2511532019, digunakan untuk menyimpan Riwayat website.

7. `Scanner input_2019 = new Scanner(System.in);`

   Untuk membaca input user

8. `int pilihan_2019;`

   variabel untuk menyimpan variabel menu

9. `do {`

   program terus berjalan sampai user pilih keluar

10. `System.out.println("\n=== Browser History NIM: 2511532019 ===");`

    Menampilkan judul program

11. `System.out.println("1. Kunjungi Website (Push)");`

    `System.out.println("2. Tombol Back (Pop)");`

    `System.out.println("3. Lihat Halaman Aktif (Peek)");`

    `System.out.println("4. Cek Status History");`

    `System.out.println("5. Keluar");`

Menampilkan menu pilihan

12. `System.out.print("Pilihan: ");`

    Meminta input dari user

13. `pilihan_2019 = input_2019.nextInt();`

    membaca angka pilihan user

14. `input_2019.nextLine();`

    enter

15. `switch (pilihan_2019) {`

    mengecek pilihan user

16. `case 1:`

    pilihan kunjungi website

17. `System.out.print("Masukkan Judul: ");`
    `String judul_2019 = input_2019.nextLine();`

    Input judul website

18. `System.out.print("Masukkan URL: ");`
    `String url_2019 = input_2019.nextLine();`

    Input URL

19. `Website_2511532019 web = new Website_2511532019(judul_2019, url_2019);`

    Membuat objek website baru

20. `history_2019.push(web);`

    menyimpan ke stack (push)

21. `System.out.println("Berhasil mengunjungi halaman!");`

    Tampilkan pesan sukses

22. `break;`

    keluar dari switch

23. `case 2:`

    pilihan tombol back (pop)

24. `if (!history_2019.isEmpty()) {`

    cek apakah stack tidak kosong

25. `Website_2511532019 removed = history_2019.pop();`

    Menghapus data paling atas

26. `System.out.println("Halaman dihapus: " + removed.getJudul_2019());`

Menampilkan halaman yang dihapus

27. `} else {`

Jika kosong

28. `System.out.println("History kosong!");`

Tampilkan pesan history kosong

29. `break;`

keluar dari switch

30. `case 3:`

pilihan lihat halaman aktif (peek)

31. `if (!history_2019.isEmpty()) {`

cek apakah history tidak kosong

32. `System.out.println("Halaman aktif: " + history_2019.peek());`

Menampilkan halaman teratas tanpa menghapusya

33. `} else {`

Jika kosong

34. `System.out.println("Tidak ada halaman aktif!");`

Tampilkan pesan tidak ada halaman aktif

35. `break;`

keluar dari switch

36. `case 4:`

pilihan cek status history

37. `if (history_2019.isEmpty()) {`

jika history kosong

38. `System.out.println("History kosong!");`

Tampilkan pesan history kosong

39. `} else {`

Jika tidak kosong

40. `System.out.println("Jumlah history: " + history_2019.size());`

Tampilkan jumlah data di stack

41. `break;`

keluar dari switch

42. `case 5:`

pilihan keluar

43. `System.out.println("Keluar dari program...");`

    Tampilkan pesan keluar dari program

44. `break;`

    keluar dari switch

45. `default:`

    jika input tidak sesuai menu

46. `System.out.println("Pilihan tidak valid!");`

    Tampilkan pesan input tidak valid

47. `while (pilihan_2019 != 5);`

    program terus berjalan sampai user pilih 5

48. `input_2019.close();`

    menutup scanner

Output

```
=== Browser History NIM: 2511532019 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!
```

```
=== Browser History NIM: 2511532019 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 3
Halaman aktif: Judul: SSO UNAND | URL: https://sso.unand.ac.id/login
```

```
=== Browser History NIM: 2511532019 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
Jumlah history: 1
```

```
=== Browser History NIM: 2511532019 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 2
Halaman dihapus: SSO UNAND
```

```
=== Browser History NIM: 2511532019 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
History kosong!
```

```
=== Browser History NIM: 2511532019 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 5
Keluar dari program...
```